

remainder 6, we concluded that 3 was the first term (a) in the continued fraction. We can take this quotient and store the result as follows:

```
import math
output = []
quotient = math.floor(big/small)
output.append(quotient)
```

In this case, we are ready to obtain a full alphabet of results (a , b , c , and so on), so we create an empty list called `output` and append our first result to it.

Finally, we have to repeat the process, just as we did for $33/6$. Remember that 33 was previously the `small` variable, but now it's the `big` one, and the remainder of our division process is the new `small` variable. Since the remainder is always smaller than the divisor, `big` and `small` will always be correctly labeled. We accomplish this switcheroo in Python as follows:

```
new_small = big % small
big = small
small = new_small
```

At this point, we have completed one round of the algorithm, and we need to repeat it for our next set of numbers (33 and 6). In order to accomplish the process concisely, we can put it all in a loop, as in [Listing 5-4](#).

```
import math
def continued_fraction(x,y,length_tolerance):
    output = []
    big = max(x,y)
    small = min(x,y)

    while small > 0 and len(output) < length_tolerance:
        quotient = math.floor(big/small)
        output.append(quotient)
        new_small = big % small
        big = small
        small = new_small
    return(output)
```

[Listing 5-4](#): *An algorithm for expressing fractions as continued fractions*